Furrow: A Fast, Original Farming-Simulation Benchmark for Long-Horizon Speedrunning

dotrdp
Independent Claude (Anthropic)
AI coding assistant

Abstract. Farming and life simulations pose long-horizon planning problems that are rare in reinforcement-learning benchmarks: success depends on seasonal windows that open once per in-game year, on weather and time-of-day gates, on tool and skill progression, and on money management, and the natural objective of human players is often a **speedrun** of a collection goal. Commercial games in this genre cannot be redistributed or run fast enough for large-scale learning. We present **Furrow**, an original farming simulation written in Rust whose objective is to complete all 24 delivery requests (79 item slots in 6 wings) of a restoration project as quickly as possible. Furrow is deterministic down to the bit, exposes temporally extended **macro** actions (15–40 decisions per in-game day) next to a primitive keyboard-level interface, rewards only speed (a per-day cost with potential-based progress shaping, discounted per in-game day), and ships a stateless scripted expert that completes the objective on 98.5% of held-out test seeds in 77.9 days on average while a masked random policy never finishes. The vectorised environment runs at 69k macro steps per second from Python on one desktop CPU. We release the simulator, Python/Gymnasium bindings, an NVIDIA NeMo Gym server for LLM agents, a trajectory format, and a SMAPI bridge to the commercial game for which Furrow is an analogue.

Keywords: reinforcement learning, imitation learning, benchmark, long horizon, speedrunning, Rust

1 Introduction

Benchmarks for sequential decision making increasingly target open-ended, long-horizon games: NetHack [1], Minecraft [2], [3], Crafter and its accelerated successor Craftax [4], [5], and procedurally generated arcade games [6]. Farming/life simulations have a different structure. Their difficulty comes less from combat or exploration than from **scheduling**: crops grow only in some seasons and must be planted early enough to mature before the season ends, some resources appear only on rainy days or at certain hours, a wandering merchant offers random stock on fixed weekdays, and investments (tool upgrades, machines, a greenhouse) pay off over weeks. Human players of such games routinely **speedrun** collection objectives — the restoration of a community building being the canonical one — which yields a crisp, scalar objective: the in-game day on which the goal is completed.

The only dedicated agent benchmark we are aware of in this genre, StarDojo [7], evaluates multimodal LLM agents inside the commercial game Stardew Valley through a SMAPI mod; it focuses on short tasks and runs at real-time speed. Learning a full-game speedrun from trajectories, and improving on it with reinforcement learning, requires millions of episodes. We therefore built Furrow: an **original** game in the same genre — all names, maps, items, numbers and the restoration objective were designed from scratch, and no code or data of any commercial game is used — implemented as a deterministic, allocation-light Rust simulator with Python bindings. A companion SMAPI [8] bridge lets the same agent interface observe and act in the player's own copy of the commercial game.

Contributions: (i) the Furrow simulator and its speedrun objective (§2); (ii) an RL interface with stable macro actions, item-centric observations and a reward whose optimum is provably pure speed (§3); (iii) a stateless scripted expert usable as a DAgger oracle and a trajectory format for imitation learning (§4); (iv) measurements of determinism, throughput and baseline difficulty (§5). A companion paper studies learning to speedrun from Furrow trajectories.

2 The game

World. Seven locations — a farm (48×36 tiles), a village with three shops and the restoration board, woods with a river, a coast, a quarry with a mine entrance, a 60-floor mine and a greenhouse that must be unlocked — connected by warps. Layouts are generated from the episode seed on a fixed topology (points of interest and warps are fixed by content; debris, trees, stumps, boulders and forage spots are procedural), with a connectivity repair pass guaranteeing that every point of interest is reachable.

Time. One tick is one game minute; days run 06:00–02:00, a season has 24 days and a year 96. Passing out at 02:00 costs energy and money. Weather (sunny, rainy, stormy, snowy) is rolled per day and tomorrow's forecast is observable; rain waters crops.

Activities. Farming (13 crops, single and regrowing, season-gated; watering, tilling, fertiliser), foraging (seasonal spawn tables in three locations), fishing (14 fish with location, season, weather and time windows and difficulty vs. skill and rod level; no reflex minigame), mining (rock kinds by depth band, pickaxe-gated ores and gems, ladders and an elevator), shops with opening hours and closed weekdays, a wandering merchant, shipping, crafting (7 recipes) and machines (furnace, kiln, pickle jar, juice press, drying rack) that turn inputs into artisan goods over minutes or days, four skills with XP and level effects, and tool upgrades that take two days.

Objective. The Old Mill has six wings with four requests each. A request lists 3–4 candidate slots `(item, count)` of which `k` must be filled (one request asks for coins instead). Completing a wing grants a reward — the greenhouse, a minecart network, +40 maximum energy, tool upgrades, a daily merchant, or money. The objective spans every activity: crops from three seasons, seasonal forage including winter items, weather/time-gated fish, ores and gems from depth, machine-made goods and 8,000 coins. A **Remixed** mode resamples slot items from per-request pools with a held-out partition for out-of-distribution evaluation.

Determinism. All randomness uses an in-crate PCG32/SplitMix64 implementation (with checked reference vectors). Exogenous events — layout, weather, forage, merchant stock, mine floors — are drawn from fresh generators keyed by `(seed, domain, day, location/floor)`, so every policy faces the **same world** on a given seed; only outcomes of actions (catches, drops, yields) consume a per-episode dynamics stream. States have a stable 64-bit hash; vectorised stepping is byte-identical to serial stepping and dataset generation is byte-identical across thread counts (tested).

3 The RL interface

Actions. The primary interface has 192 temporally extended macro actions enumerated from content in a stable order: `GoTo(location)`, `Plant(crop, farm|greenhouse)`, `WaterCrops`, `Harvest`, `ClearField`, `Forage(location)`, `Fish(spot)`, `MineDescend`, `MineFloor`, `ChopWood`, `ChopHardwood`, `Buy(item, 1|5|25)`, `ShipSurplus`, `Craft(recipe)`, `LoadMachines`, `CollectMachines`, `UpgradeTool(tool)`, `Deliver`, `PayCoins`, `Eat`, `Wait30` and `Sleep`. Each is executed by a deterministic controller that issues primitive steps with BFS path finding, is time-boxed (120 minutes of activity after travel), and costs at least five minutes even when it fails, so no policy can stall the clock. Cheap preconditions computed with travel-time estimates (shop open on arrival, affordable, reachable, inputs held) define an action mask; at most 64 decisions are accepted per day. A primitive interface (67 keyboard-level actions plus a 7-channel egocentric grid) is available for hierarchical work; every primitive advances the clock by at least one minute.

Observations. A 777-dimensional vector whose layout depends only on content: calendar and weather (with forecast), player state, **item-centric** features for each of the 85 items (held, still needed by open requests, coverage, scarcity, today's best price), request/wing progress, travel times to every point of interest, crop aggregates (planted, watered, ready, days to maturity, maturing before season end), machine states, forage availability, shop and merchant status. Item-centric features keep their meaning under Remixed objectives. A compact deterministic text description (`describe()`) serves LLM agents.

Reward. Speed is the only objective: a cost of −1 per day boundary crossed, plus potential-based shaping [9] on progress (slots, requests, wings) with $\Phi(s')$ discounted by $\gamma_{\text{day}}^{\text{day_ends}}$. Discounting is per in-game day, never per decision, so policies cannot profit from padding days with extra decisions; with this choice the expected discounted return is an increasing function of $\mathbb{E}[\gamma^T]$, where T is the completion day.

Interfaces. `furrow.Env`/`furrow.VecEnv` (PyO3, GIL released while stepping) with a Gymnasium-style API that accepts action ids or names; Gymnasium [10] registrations (`Furrow-Macro-v1`, `Furrow-Primitive-v1`, `Furrow-Remixed-v1`) passing the official environment checker, with a vector environment that declares `SAME_STEP` autoreset and reports terminal observations; and an NVIDIA NeMo Gym [11] resources server that exposes the game to LLM agents as tools.

4 Expert and data

The scripted expert is a stateless priority policy: it chooses, for every open request, the `k` candidate slots with the lowest estimated acquisition cost (a recursive cost model over crops and their seasonal windows, forage seasons, fish windows and rod gates, mine depth and pickaxe gates, shops, the merchant, recipes and machines), expands them into intermediate needs (bars, machine inputs, tool upgrades, coins), and then returns the first unmasked macro of an ordered rule list (sleep before passing out, deliver completions, tend crops and machines, buy, plant, upgrade, time-gated fishing, seasonal foraging, mining, chopping, money). Because it is a function of the current state only, it can label states visited by a learner (DAgger [12]). Noise injection with clean labels (DART [13]) and seven priority **styles** yield diverse demonstrations.

Trajectories are stored as `furrow-traj/2` directories of memory-mappable `.npy` shards (64-byte aligned, f16 observations, bit-packed masks, executed and expert actions, day boundaries, minutes, progress counters, expert intent), written atomically and episode-aligned.

5 Measurements

Envs	Policy	Macro steps / s
16	random	28,867
16	expert	36,751
64	random	49,591
64	expert	65,736
256	random	68,691
256	expert	85,864

Table 1: Macro-step throughput of `furrow.VecEnv` driven from Python on one desktop CPU (AMD Ryzen 7 9700X, 8 cores/16 threads), including observation/mask copies into NumPy and, for "expert", expert labelling of every environment. On expert trajectories a macro step advances the game by 48 game minutes on average within a day (each game minute is at least one simulated primitive step), with about 16 decisions per in-game day.

The scripted expert finishes all 24 requests on 98.5% of 200 held-out test seeds with a mean censored completion day of 77.9; a masked uniform-random policy completes 0.5% within the 192-day cap. Completion days are multimodal: a missed seasonal window delays completion by up to a year (companion paper, Fig. 1). Adding 10% random actions to the expert **shortens** average completion (66 vs. 79 days on disjoint training seeds), showing that the expert leaves headroom that learning can exploit.

Objective	Policy	Success	Mean C	IQM C
Fixed	expert	0.985	77.9	69.9
Fixed	random	0.005	192.0	192.0
Remixed (train pools)	expert	0.965	93.1	83.1
Remixed (train pools)	random	0.000	192.0	192.0
Remixed (held-out pools)	expert	0.660	114.6	105.2
Remixed (held-out pools)	random	0.005	192.0	192.0

Table 2: Difficulty tiers on 200 held-out test seeds. **Remixed** resamples the item of every request slot from per-request pools; the held-out pools contain items never requested in the Fixed objective. The same expert, tuned on the Fixed objective, degrades sharply on held-out pools, which makes them a generalisation test for learned policies.

6 Related work

Fast simulators — EnvPool [14], Sample Factory [15] and JAX environments such as Craftax [5] — made large-scale RL on games routine. Furrow follows the same principles (pre-allocated batch buffers, a per-instance thread pool, no copies under the interpreter lock) in Rust. Long-horizon imitation benchmarks in NetHack [16], [17] and Minecraft [2] show that behaviour cloning from large datasets recovers only part of the demonstrator's competence; LLM-agent suites such as BALROG [18] and StarDojo [7] report low success on multi-day tasks. Long single-game RL efforts such as Pokémon Red [19] share our focus on one long objective. Our evaluation follows recommendations for statistically robust reporting [20], [21].

7 Limitations

Furrow is an analogue, not a replica: results do not transfer to the commercial game without retraining, and the bridge supports evaluation and data collection at real-time speed only. The expert reads quantities that are present in the observation but computes them from the full state; a strict observation-only expert is future work. Primitive-level control is provided but not yet benchmarked.

Bibliography

[1] H. Küttler *et al.*, "The NetHack Learning Environment," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] B. Baker *et al.*, "Video PreTraining (VPT): Learning to Act by Watching Unlabeled Online Videos," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[3] L. Fan *et al.*, "MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.

[4] D. Hafner, "Benchmarking the Spectrum of Agent Capabilities," in *International Conference on Learning Representations (ICLR)*, 2022.

[5] M. Matthews *et al.*, "Craftax: A Lightning-Fast Benchmark for Open-Ended Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2024.

[6] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, "Leveraging Procedural Generation to Benchmark Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2020.

[7] W. Tan and others, "StarDojo: Benchmarking Open-Ended Behaviors of Agentic Multimodal LLMs in Production-Living Simulations with Stardew Valley," *arXiv preprint arXiv:2507.07445*, 2025.

[8] Pathoschild, "SMAPI: the Stardew Valley modding API." 2026.

[9] A. Y. Ng, D. Harada, and S. Russell, "Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999.

[10] M. Towers *et al.*, "Gymnasium: A Standard Interface for Reinforcement Learning Environments," *arXiv preprint arXiv:2407.17032*, 2024.

[11] NVIDIA, "NeMo Gym: build RL environments for LLM training." 2026.

[12] S. Ross, G. J. Gordon, and J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[13] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "DART: Noise Injection for Robust Imitation Learning," in *Conference on Robot Learning (CoRL)*, 2017.

[14] J. Weng *et al.*, "EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.

[15] A. Petrenko, Z. Huang, T. Kumar, G. Sukhatme, and V. Koltun, "Sample Factory: Egocentric 3D Control from Pixels at 100000 FPS with Asynchronous Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2020.

[16] E. Hambro *et al.*, "Dungeons and Data: A Large-Scale NetHack Dataset," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.

[17] U. Piterbarg, L. Pinto, and R. Fergus, "NetHack is Hard to Hack," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[18] D. Paglieri and others, "BALROG: Benchmarking Agentic LLM and VLM Reasoning On Games," in *International Conference on Learning Representations (ICLR)*, 2025.

[19] M. Pleines, D. Addis, D. Rubinstein, F. Zimmer, M. Preuss, and P. Whidden, "Pokemon Red via Reinforcement Learning," *arXiv preprint arXiv:2502.19920*, 2025.

[20] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[21] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep Reinforcement Learning that Matters," in *AAAI Conference on Artificial Intelligence*, 2018.